

Web and Mobile Application Security

Final Project

**Identifying, Exploiting, and Securing a Vulnerable Application Against
XSS, CSRF, and SQL Injection**

Table of Contents

Introduction	3
Setting Up the Environment.....	3
Project Requirements	4
Part 1: XSS (Cross-Site Scripting).....	4
Exploitation of XSS.....	4
Implementation of a Countermeasure:	6
Validation:	7
Part 2: CSRF (Cross-Site Request Forgery)	10
Exploitation of CSRF:	10
Implementation of a Countermeasure:	12
Validation:	14
Part 3: SQL Injection:.....	15
Exploitation of SQL Injection:	15
Implementation of a Countermeasure:	16
Validation:	17
Conclusion	19

Introduction

Modern web applications can be targets for attackers who try to take advantage of weaknesses in their design or coding. Understanding these vulnerabilities and learning how to fix them is very important for anyone studying cybersecurity or web development. In this project, we used **Damn Vulnerable Web Application (DVWA)**, a purposely insecure platform, to explore three common web attacks: **XSS, CSRF, and SQL Injection**.

Using a Docker-based setup, we were able to see how attackers can exploit these vulnerabilities to manipulate the application, steal data, or perform actions without permission. The project also focused on applying security measures, like adding a **Content Security Policy (CSP)**, using **CSRF tokens**, and implementing **prepared statements** for database queries. By testing the application before and after adding these protections, we learned why secure coding practices are so important.

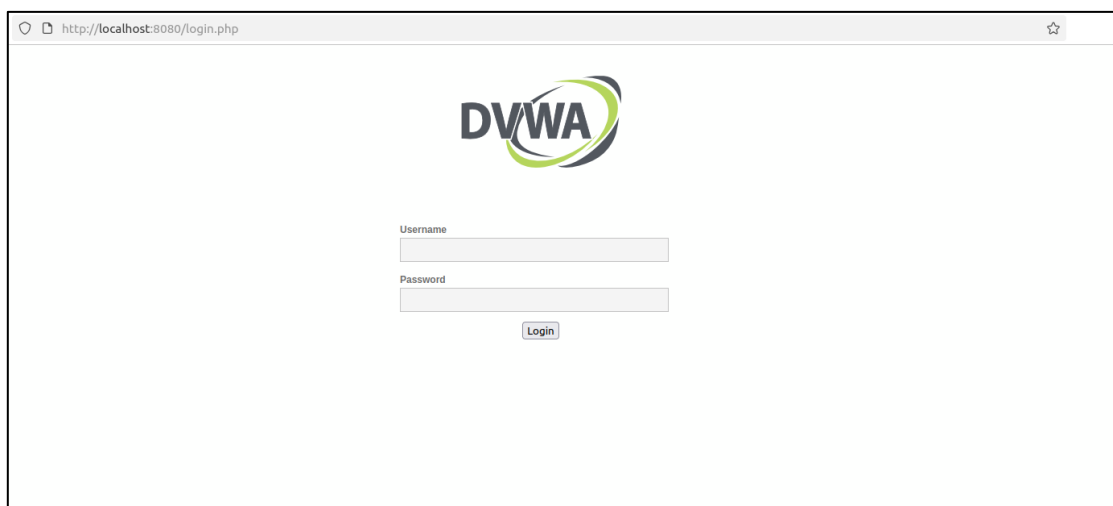
Overall, working on this project gave us hands-on experience with both attacking and defending web applications, which helped us understand web security in a practical way.

Setting Up the Environment

I followed the steps given by the project document and successfully run the container and opened the DVWA website.

- Step 1: Build and Run the Application

I used the Dockerfile given and run the commands “docker build -t dvwa-project .” and “docker run -d -p 8080:80 dvwa-project” to access the application through the browser at: <http://localhost:8080>



- Step 2: Configure DVWA

I logged in to the web application using the default username admin and password password, then I edited the security level to be low.

Setup DVWA

Instructions

About

Database Setup

Click on the 'Create / Reset Database' button below to create or reset your database.
If you get an error make sure you have the correct user credentials in: `/var/www/html/config/config.inc.php`

If the database already exists, **it will be cleared and the data will be reset**.
You can also use this to reset the administrator credentials ("admin / password") at any stage.

Setup Check

Operating system: `*nix`
Backend database: **MySQL**
PHP version: **7.0.30-0+deb9u1**

Web Server SERVER_NAME: `localhost`

PHP function display_errors: **Disabled**
PHP function safe_mode: **Disabled**
PHP function allow_url_include: **Disabled**
PHP function allow_url_fopen: **Enabled**
PHP function magic_quotes_gpc: **Disabled**
PHP module gd: **Installed**
PHP module mysql: **Installed**
PHP module pdo_mysql: **Installed**

MySQL username: `app`
MySQL password: `*****`
MySQL database: `dvwa`
MySQL host: `127.0.0.1`

reCAPTCHA key: **Missing**

[User: www-data] Writable folder /var/www/html/hackable/uploads/: **Yes**
[User: www-data] Writable file /var/www/html/external/phpids/0.6/lib/IDS/tmp/phpids_log.txt: **Yes**

[User: www-data] Writable folder /var/www/html/config: **Yes**
Status in red, indicate there will be an issue when trying to complete some modules.

If you see disabled on either `allow_url_fopen` or `allow_url_include`, set the following in your php.ini file and restart Apache.

`allow_url_fopen = On`
`allow_url_include = On`

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

DVWA Security

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Low

Submit

PHPIDS

PHPIDS v0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

PHPIDS works by filtering any user supplied input against a blacklist of potentially malicious code. It is used in DVWA to serve as a live example of how Web Application Firewalls (WAFs) can help improve security and in some cases how WAFs can be circumvented.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently: **disabled**. [\[Enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Security level set to low

Project Requirements

Part 1: XSS (Cross-Site Scripting)

Exploitation of XSS

My goal here was to trick the website into running my malicious code. I found two ways the application was vulnerable:

- **Reflected XSS :**

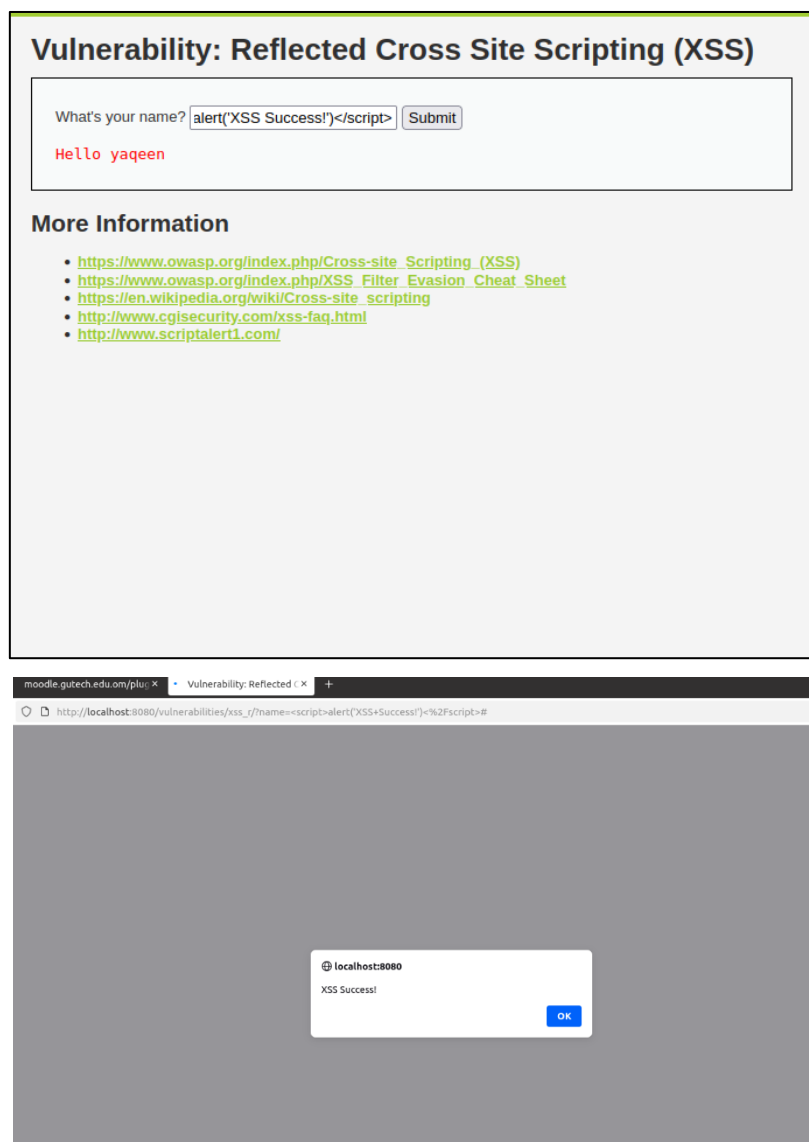
- **How we Hacked It:** we put a piece of code "<script>alert('XSS Success!')</script>" into What's your name box.
- **What Happened:** When we clicked 'Submit', the website immediately grabbed my code and printed it back onto the page. The browser saw the <script> tag and ran it, making a pop-up box appear.

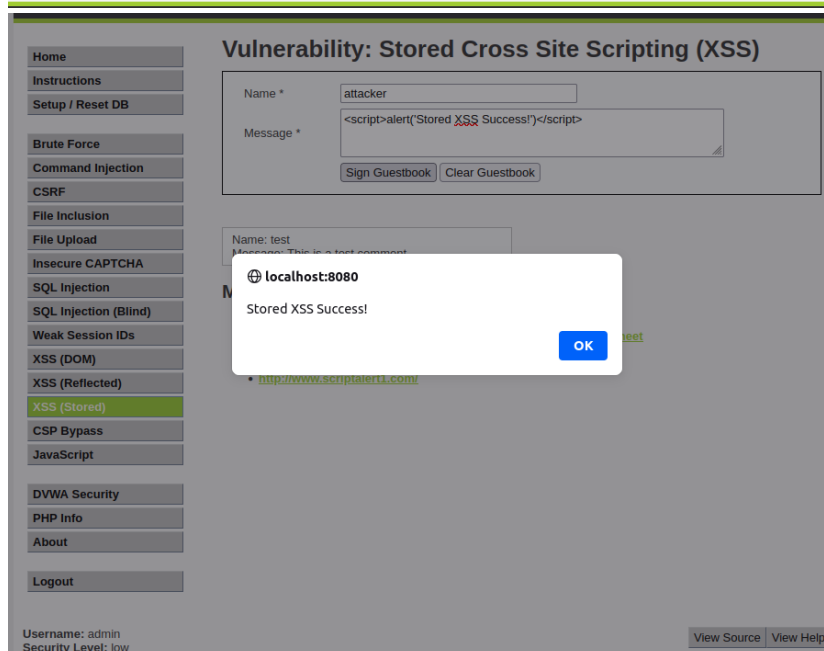
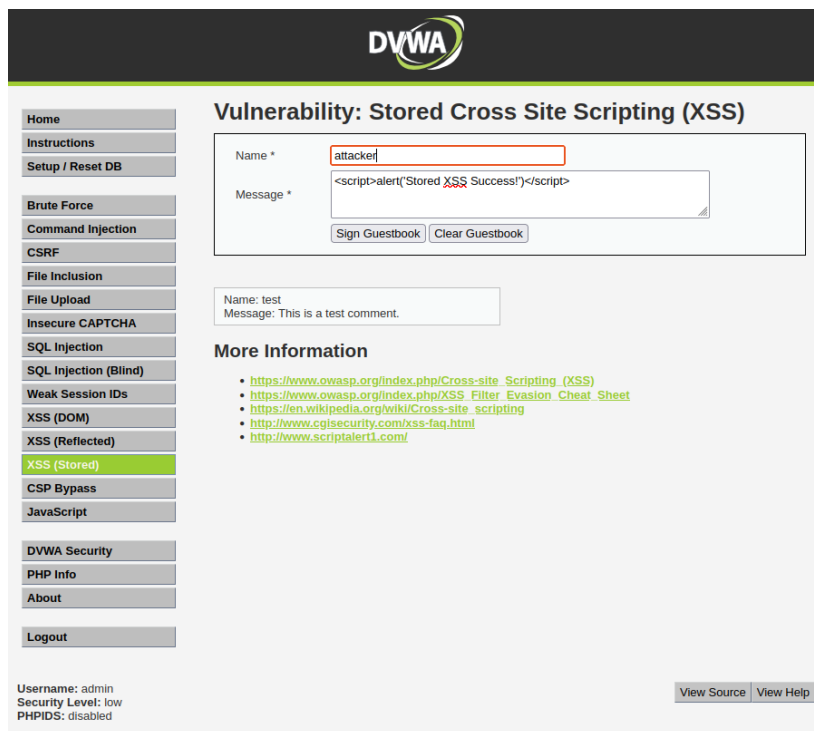
Basically, it is a simple XSS reflection that takes an input and prints it in the HTML page, and due to no filter, it executed the code and showed a pop-up box.

- **Stored XSS:**

- **How we Hacked It:** we used the same code “<script>alert('XSS Success!')</script>” but put it into the 'Message' field of the Guestbook (Stored XSS page).
- **What Happened:** The website saved my malicious code in its database. When we or anyone else loaded that page, the code was pulled from the database and executed, making the pop-up appear for everyone.

Stored XSS occurs because DVWA saves my input into the database without sanitizing, then prints it back to every visitor, causing the browser to execute the script automatically. And this is more dangerous than Reflected XSS because it persists, affects every user, and does not require them to click or submit anything.





Implementation of a Countermeasure:

The goal here was to stop the website from running our malicious code. First, we entered the Docker container and added the CSP header to dvwaPage.inc.php “nano dvwaPage.inc.php” , then we restarted the Apache and validated.

The security header blocks the inline scripts and “<script>alert('XSS')</script>” and allows only JavaScript files of code hosted only by the same server.

```
seed@VM: ~/dvwa-project
GNU nano 2.7.4 File: dvwaPage.inc.php Modified

<?php
header("Content-Security-Policy: default-src 'self'; script-src 'self';");
if( !defined( 'DVWA_WEB_PAGE_TO_ROOT' ) ) {
    die( 'DVWA System error- WEB_PAGE_TO_ROOT undefined' );
    exit;
}

session_start(); // Creates a 'Full Path Disclosure' vuln.

if ( !file_exists(DVWA_WEB_PAGE_TO_ROOT . 'config/config.inc.php')) {
    die ("DVWA System error - config file not found. Copy config/config.inc$
}

// Include configs
require_once DVWA_WEB_PAGE_TO_ROOT . 'config/config.inc.php';
require_once( 'dvwaPhpIds.inc.php' );

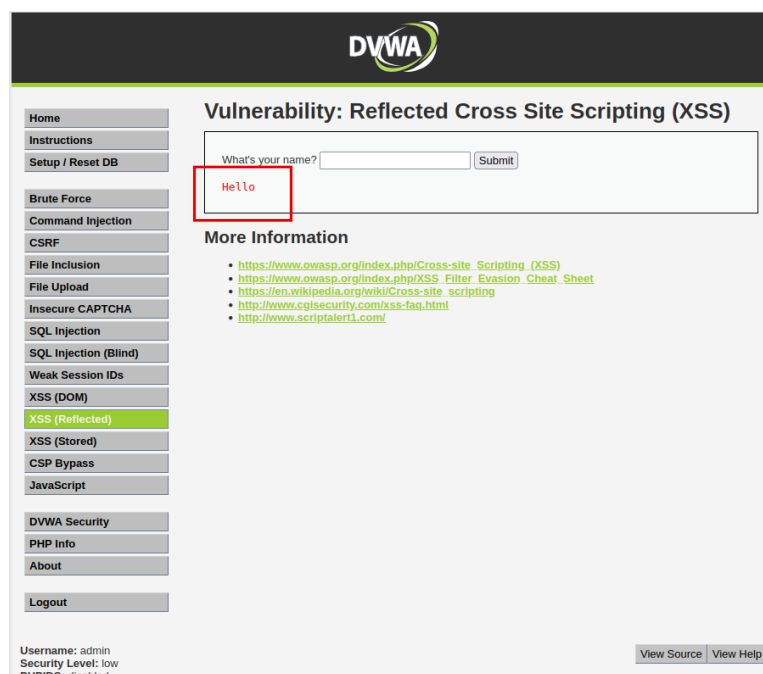
// Declare the $html variable

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^\ Replace ^U Uncut Text ^T To Spell ^_ Go To Line
```

Validation:

We have tested the reflected and the stored XSS attack by entering the same script “**<script>alert('XSS');</script>**” and both attacks didn’t happen (were blocked) and no pop-up box appeared, because the browser blocked the inline scripts.

- In the reflected XSS, we entered the script **<script>alert('XSS');</script>** and it didn’t run and show space next to Hello.



DVWA

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
DVWA Security
PHP Info
About
Logout

Vulnerability: Stored Cross Site Scripting (XSS)

Name *
Message *

Sign Guestbook Clear Guestbook

Name: test
Message: This is a test comment.

Name: test
Message:

More Information

- [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))
- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

View Source View Help

Username: admin
Security Level: low
PHPIDS: disabled

- In the stored XSS, in the name we entered **test**, and in the message, we put the script “**<script>alert('XSS');</script>**”. And it didn’t run and printed nothing in the message.

Securing DVWA with CSP Using Another Approach

To apply the Content Security Policy inside the container, I used an alternative method by modifying the Apache configuration directly. After restarting the Apache service using:

service apache2 restart

I created a new configuration file named **csp.conf** using the following command:

```
echo 'Header always set Content-Security-Policy "default-src \"self\""; script-src \"self\"\"\"\" > /etc/apache2/conf-enabled/csp.conf
```


This command instructs Apache to send a CSP header with every server response. The applied policy restricts all content to originate from the same domain ('self') and specifically limits JavaScript execution to scripts hosted on the same server. This effectively blocks any external or injected scripts.

```
[Wed Dec 10 07:03:52.155795 2025] [mpm_prefork:notice] [pid 296] AH00169: caught SIGTERM, shutting down
root@c544d5a15b2d:/# a2enmod headers
Enabling module headers.
To activate the new configuration, you need to run:
  service apache2 restart
root@c544d5a15b2d:/# service apache2 restart
[...] Restarting Apache httpd web server: apache2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
. ok
root@c544d5a15b2d:/# echo 'Header always set Content-Security-Policy "default-src \'\'self\'\'"; script-src \'\'self\'\'"' > /etc/apache2/conf-enabled/csp.conf
root@c544d5a15b2d:/# service apache2 restart
[...] Restarting Apache httpd web server: apache2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message
. ok
root@c544d5a15b2d:/#
```

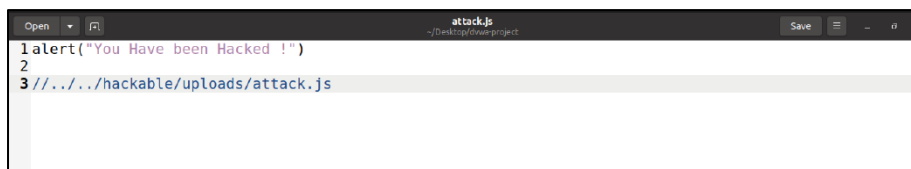
CSP Testing

To test the effectiveness of the implemented CSP, we attempted to inject a malicious JavaScript payload. A file named **attack.js** containing:

```
alert("You Have been Hacked !");
```

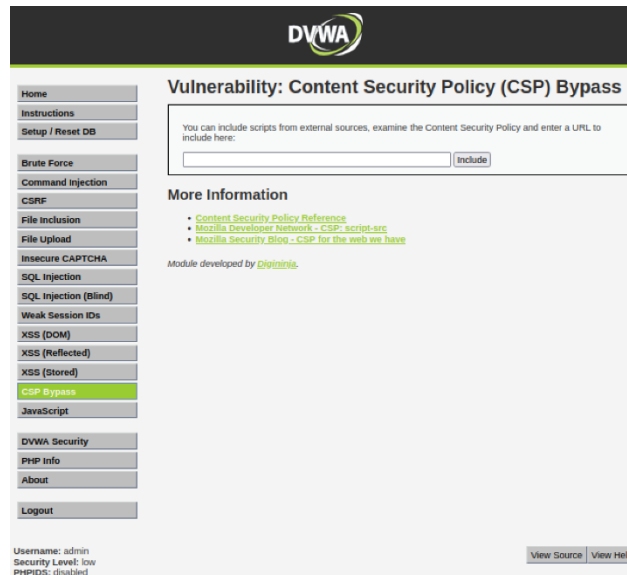
Was uploaded to:

```
../../hackable/uploads/attack.js
```



I then tried to include this script through various methods, including CSP bypass techniques. However, the script **did not execute**, and no alert dialog was triggered.

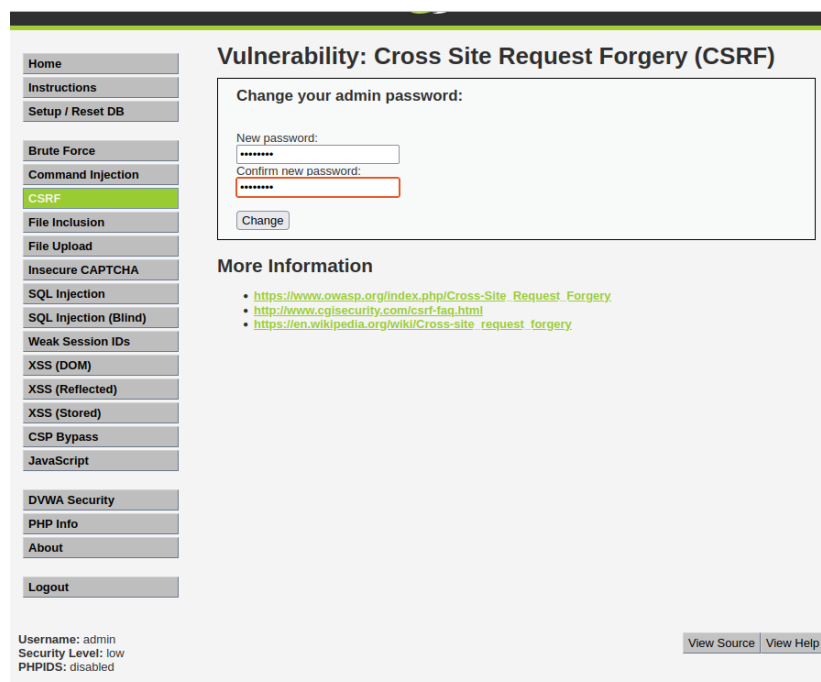
The failure of the injected script to run confirms that the CSP configuration works as intended. The policy successfully prevents the browser from executing unauthorized scripts, even when they reside on the server or when bypass attempts are made. This demonstrates that the implemented CSP is an effective countermeasure against XSS attacks.



Part 2: CSRF (Cross-Site Request Forgery)

Exploitation of CSRF:

This form is vulnerable because it accepts password-change requests from any external webpage, even if the user did not click it intentionally. The goal here was to trick the website into running my malicious code. First, I tried to change the password to see how it works. I changed it to admin123, then it showed a message “password changed”.



Then I created an HTML code to do the attack, because the CSRF password changing vulnerable page allows the user to change from any external website. The code creates a password and submits it to request a password change to DVWA. I knew the exact way how the DVWA form works by Right-click → inspect, then write the correct code according to that.

```
GNU nano 4.8      csrf_attack.html
<html>
<body>
  <h3>CSRF Attack Demo</h3>

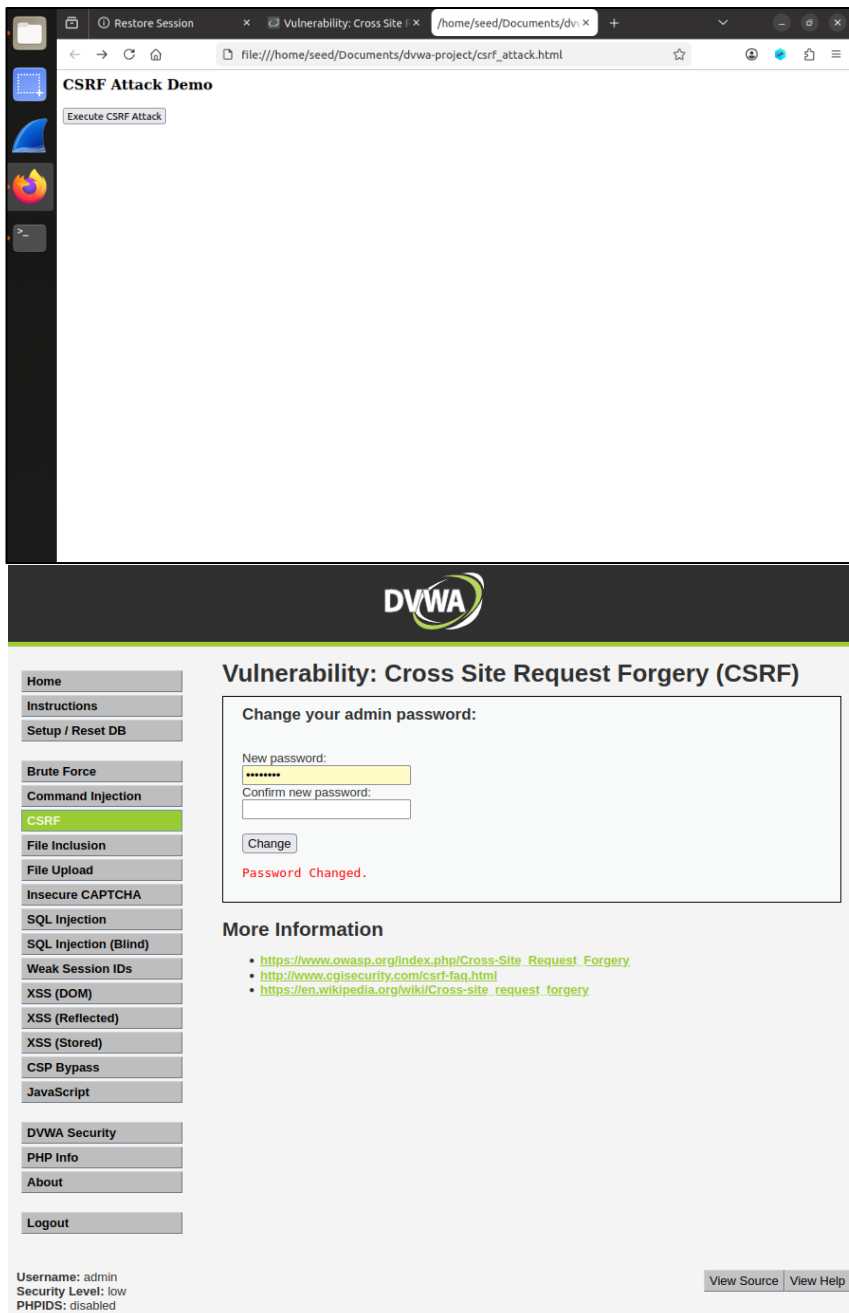
  <form action="http://localhost:8080/vulnerabilities/csrf/" method="GET">
    <input type="hidden" name="password_new" value="new123">
    <input type="hidden" name="password_conf" value="new123">
    <input type="hidden" name="Change" value="Change">
    <input type="submit" value="Execute CSRF Attack">
  </form>

</body>
</html>
```

[Read 14 lines]

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line

I have opened the **csrf_attack.html** I created in my browser and made sure I was logged into DVWA at the same time, then clicked the button Execute CSRF Attack, and it successfully changed the password from another page.



Implementation of a Countermeasure:

After exploiting the CSRF attack, I have configured the countermeasure to secure the page. First, I navigated to the DVWA container and went to the `/var/www/html/dvwa/vulnerabilities/csrf` directory to find the `index.php` file.

```

seed@VM: ~/dvwa-project
root@9579f7f76edb:/var/www/html/vulnerabilities# cd vulnerabilities/
bash: cd: vulnerabilities: No such file or directory
root@9579f7f76edb:/var/www/html/vulnerabilities# ls
brute  exec      sql_i_blind  view_source_all.php  xss_s
captcha  fi        upload      weak_id
csp      javascript view_help.php  xss_d
csrf     sql_i     view_source.php  xss_r
root@9579f7f76edb:/var/www/html/vulnerabilities# cd vulnerabilities/
bash: cd: vulnerabilities: No such file or directory
root@9579f7f76edb:/var/www/html/vulnerabilities# cd /vulnerabilities/
bash: cd: /vulnerabilities: No such file or directory
root@9579f7f76edb:/var/www/html/vulnerabilities# cd /vulnerabilities/
bash: cd: /vulnerabilities: No such file or directory
root@9579f7f76edb:/var/www/html/vulnerabilities# ls
brute  exec      sql_i_blind  view_source_all.php  xss_s
captcha  fi        upload      weak_id
csp      javascript view_help.php  xss_d
csrf     sql_i     view_source.php  xss_r
root@9579f7f76edb:/var/www/html/vulnerabilities# cd csrf/
root@9579f7f76edb:/var/www/html/vulnerabilities/csrf# ls
help index.php source
root@9579f7f76edb:/var/www/html/vulnerabilities/csrf# nano source
root@9579f7f76edb:/var/www/html/vulnerabilities/csrf# cd source
root@9579f7f76edb:/var/www/html/vulnerabilities/csrf/source# ls
high.php impossible.php low.php medium.php
root@9579f7f76edb:/var/www/html/vulnerabilities/csrf/source# nano
low.php
root@9579f7f76edb:/var/www/html/vulnerabilities/csrf/source#

```

After finding the index.php file /var/www/html/dvwa/vulnerabilities/csrf/index.php I used the command nano index.php to make the following configurations:

CSRF token was added. This ensures that only legitimate requests originating from the DVWA site itself are accepted, by adding a CSRF token generator at the top of the index.php file

“ **session_start();**

if (!isset(\$_SESSION['csrf_token'])) {

\$_SESSION['csrf_token'] = bin2hex(random_bytes(32)); } “

This will create a random token that will be stored in the user session.

```

GNU nano 2.7.4      File: index.php
<?php
session_start();

// Generate token if not exists
if (!isset($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

define('DVWA_WEB_PAGE_TO_ROOT', '../..');
require_once DVWA_WEB_PAGE_TO_ROOT . 'dvwa/includes/dvwaPage.inc.php';

dvwaPageStartup(array('authenticated', 'phpids'));

```

And the token was added to the html form “**<input type="hidden" name="csrf_token" value="<?php echo \$_SESSION['csrf_token']; ?>"**”. This will make sure that each password change will contain a token.

```

        <form action="#" method="GET">
            <input type="hidden" name="csrf_token" value="{$_SES$
if ($vulnerabilityFile == 'impossible.php') {
    $page['body'] .= "
        Current password:<br />
        <input type="password" AUTOCOMPLETE="off" name="pass$
}
$page['body'] .= "
    New password:<br />
    <input type="password" AUTOCOMPLETE="off" name="pass$
    Confirm new password:<br />
    <input type="password" AUTOCOMPLETE="off" name="pass$
    <br />
    <input type="submit" value="Change" name="Change">
</form>

```

Then I also added a token checking code in the low.php file to check the token before changing the password.

```

<?php
if( isset( $_GET[ 'Change' ] ) ) {
    if ( !isset( $_GET[ 'csrf_token' ] ) || $_GET[ 'csrf_token' ] !=$_
        die("<pre>CSRF token validation failed!</pre>");
    // Get input
    $pass_new = $_GET[ 'password_new' ];
    $pass_conf = $_GET[ 'password_conf' ];

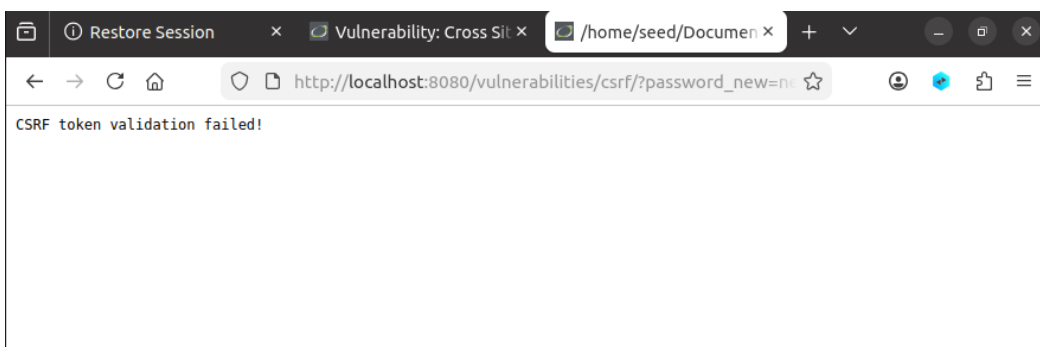
    // Do the passwords match?
    if( $pass_new == $pass_conf ) {
        // They do!
        $pass_new = ((isset($_GLOBALS[ "__mysqli_ston" ])) &$
            $pass_new = md5( $pass_new );

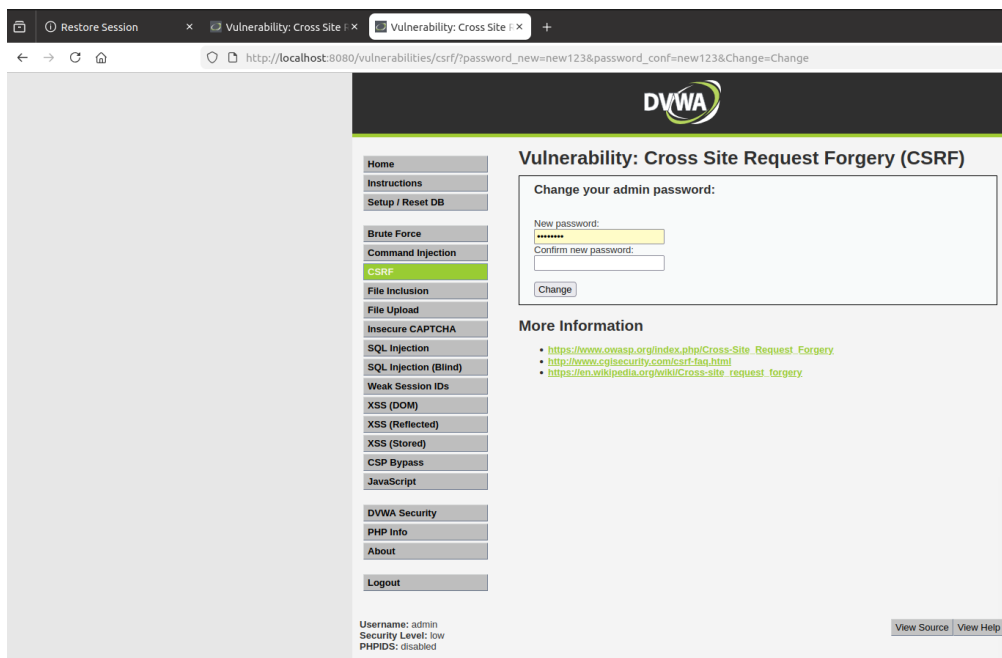
        // Update the database
        $insert = "UPDATE `users` SET password = '$pass_n$
        $result = mysqli_query($_GLOBALS[ "__mysqli_ston" ])$

```

Validation:

I have successfully validated that the countermeasures work perfectly by navigating to the **malicious HTML** I have created to do the attack, and tried to do the attack again, and failed to change the password.

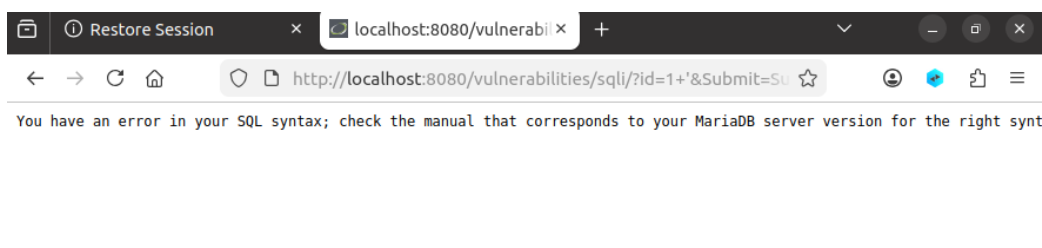




Part 3: SQL Injection:

Exploitation of SQL Injection:

As I want to extract information from the database using SQL injection, I navigate to the vulnerable SQL injection page in DVWA and first to make sure that the inputs go directly to the database I tried “1;” and get a syntax error, and then type the simple “1’ OR 1=1” and the attack in successfully happened it shows the users information from the database. I used “1’” to close the userid “1’” and then but “OR 1=1 ” so the condition statement will always run the other condition if the first is false, “–” used to tell the SQL to ignore the rest statements.



User ID:

ID: 1' OR 1=1 #
First name: admin
Surname: admin

ID: 1' OR 1=1 #
First name: Gordon
Surname: Brown

ID: 1' OR 1=1 #
First name: Hack
Surname: Me

ID: 1' OR 1=1 #
First name: Pablo
Surname: Picasso

ID: 1' OR 1=1 #
First name: Bob
Surname: Smith

Implementation of a Countermeasure:

To implement the SQL Injection countermeasure, I first navigated to the file `/var/www/html/vulnerabilities/sqli/source/low.php` and examined the original SQL code:

```
$id = $_REQUEST['id'];
```

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

```
$result = mysqli_query($GLOBALS["__mysqli_ston"], $query);
```

This code is vulnerable because the user input `$id` is inserted directly into the SQL query. An attacker can easily modify the query using payloads such as `1 OR 1=1`, which results in unauthorized data disclosure.

To fix this issue, I replaced the insecure code with a secure version that uses prepared statements:

```
"$db = $GLOBALS["__mysqli_ston"];
```

```
$stmt = mysqli_prepare($db,
```

```
    "SELECT first_name, last_name FROM users WHERE user_id = ?");
```

```
mysqli_stmt_bind_param($stmt, "i", $id);
```

```
mysqli_stmt_execute($stmt);
```

```
mysqli_stmt_bind_result($stmt, $first, $last);
```

```
while (mysqli_stmt_fetch($stmt)) {
```

```
    $html .= "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
```



```
mysqli_stmt_close($stmt);
```

```
mysqli_close($db);”
```

In this version, the ? placeholder is used instead of directly inserting the variable. This means the input is bound safely using **mysqli_stmt_bind_param()**, which forces the value to be treated as data, not SQL code. As a result, attacks like 1 OR 1=1 no longer work, and the query becomes secure.

```
<?php
if( isset( $_REQUEST[ 'Submit' ] ) ) {

    // Get input
    $id = $_REQUEST['id'];

    // Database connection
    $db = $GLOBALS["__mysqli_ston"];

    // SECURE: Prepared Statement
    $stmt = mysqli_prepare($db,
        "SELECT first_name, last_name FROM users WHERE user_id = ?"
    );

    // Bind parameter (integer)
    mysqli_stmt_bind_param($stmt, "i", $id);

    // Execute
    mysqli_stmt_execute($stmt);

    // Bind results
    mysqli_stmt_bind_result($stmt, $first, $last);

    // Fetch results
    while (mysqli_stmt_fetch($stmt)) {
        $html .= "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
    }
}
```

Prepared statements prevent SQL Injection because:

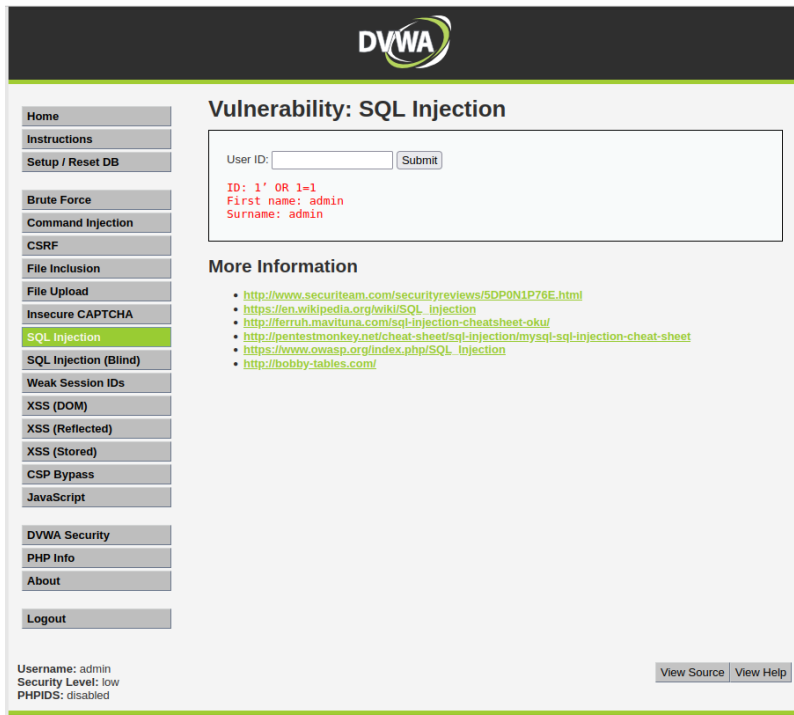
- User input cannot change the structure of the SQL query
- The database treats input as data, not commands
- It blocks malicious SQL payloads

Validation:

After applying the secure code, I tested the application again using common SQL injection payloads to ensure that malicious inputs were no longer processed as part of the SQL query. When entering this payload:

“' OR '1'='1'”

The system displayed only the normal Admin record (**First Name: admin, Surname: admin**). This occurred because the injected value is no longer accepted as a valid integer, and the query defaults to using the standard **ID value of 1**. This confirms that the input is handled safely and the injected condition is not executed.



I also tested a UNION-based payload:

“ UNION SELECT 1,2,version() – “

This resulted in a blank page or an error message, and no database information was revealed. The system rejected the input because it does not match the expected numeric format for the parameter.



Validation Results

- SQL injection attempts are no longer executed.
- Invalid inputs are ignored or rejected, preventing malicious behavior.
- Only valid and expected data (ID = 1) is returned.
- No unauthorized database information is exposed.
- The prepared statement countermeasure is functioning effectively.

Conclusion

This project provided a complete path through identifying, exploiting, and securing common web vulnerabilities within DVWA. By performing attacks such as XSS, CSRF, and SQL Injection, the practical risks and real-world impact of insecure applications became clear. Implementing countermeasures, including CSP headers, CSRF tokens, and prepared SQL statements, demonstrated how proper defensive measures can effectively block these attacks.

Working on this project together helped us understand key cybersecurity concepts, like the importance of secure coding, input validation, and good application design. We saw firsthand how vulnerabilities in web applications can be exploited, and how countermeasures can prevent attacks. Testing each defense as a team showed us that applying standard security practices really makes a web application much safer.